



HORIZON: Estimating Alias Analysis Precision Bounds and Their Impact on Performance

Khushboo Chitre

IIIT Delhi

India

khushbooc@iiitd.ac.in

Piyus Kedia

IIIT Delhi

India

piyus@iiitd.ac.in

Rahul Purandare

University of Nebraska-Lincoln

USA

rahul@unl.edu

Abstract

Alias analysis is a technique to identify whether a memory location can be accessed in more than one way. An ideal alias analysis implementation should be both precise and scalable. However, in practice, implementations of alias analysis have to make a trade-off between precision and scalability. The alias analysis implementations that perform inter-procedural analysis are more precise (answer a higher number of alias queries with certainty), but expensive, making them infeasible for practical use. Most compiler developers opt for intra-procedural analysis over inter-procedural, thereby compromising the precision of alias analysis implementations to achieve scalability. For compilers, this compromise leads to a loss in optimization opportunities, limiting the performance achievable by the compiled program.

In this work, we present HORIZON, a tool that estimates the upper bound on alias analysis precision improvement and its impact on program performance, enabling an understanding of how compromised precision affects execution. HORIZON implements a profiling-based approach to gather precise alias information (must- and no-alias) missed by existing alias implementations and provides this information to the compiler to estimate potential performance gains. Integrated with LLVM, we applied HORIZON to evaluate the default LLVM alias analysis on standard benchmarks, demonstrating that precision improvement could range from 0% to 33%, with corresponding performance impact between 0% and 53%, indicating the effectiveness of our approach and tool.

CCS Concepts: • Software and its engineering → Compilers.

Keywords: compiler optimization, performance, alias analysis, profiling

ACM Reference Format:

Khushboo Chitre, Piyus Kedia, and Rahul Purandare. 2026. HORIZON: Estimating Alias Analysis Precision Bounds and Their Impact



This work is licensed under a Creative Commons Attribution 4.0 International License.

CC '26, Sydney, NSW, Australia

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2274-5/2026/01

<https://doi.org/10.1145/3771775.3786270>

on Performance. In *Proceedings of the 35th ACM SIGPLAN International Conference on Compiler Construction (CC '26)*, January 31 – February 1, 2026, Sydney, NSW, Australia. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3771775.3786270>

1 Introduction

The use of pointers in languages such as C or Java has led to a lot of research opportunities in this field [30]. Many program analysis techniques [4, 39, 50] use information related to pointers for analyzing programs. These analyses particularly rely on the knowledge of whether two pointers (pointer pair) can point to the same memory location during execution, i.e., whether they are *aliases*. An analysis that extracts alias relationship information for pointer pairs is called alias analysis.

The quality of a static alias analysis implementation is measured in terms of scalability and precision. Inter-procedural alias analyses [11, 12, 16, 18, 20, 22, 25–29, 31, 34, 36, 37, 42–44, 46–48, 51, 53–58, 60, 61, 64, 65] are more precise as compared to intra-procedural alias analysis [21]. However, these are more complex and less efficient. As a result, their more precise versions often do not scale to real-world applications. Since scalability is non-negotiable, most compiler developers avoid using inter-procedural alias analyses in practice, opting for intra-procedural analyses instead. Such analyses miss out on useful alias information related to function arguments and return values which could potentially be used to trigger additional optimizations. As a result, the performance of a program is compromised.

One of the key challenges in alias analysis is understanding whether increasing its precision leads to meaningful performance improvements. A common approach to estimating the potential of improved alias analysis is to make optimistic assumptions for the pointers. Chowdhury et al. [17] try to establish a loose upper bound on optimization opportunities by implementing a static alias analysis algorithm that assumes the no-alias relationship for all pointer pairs by default. This highlights how conservative aliasing assumptions in real-world compilers can inhibit scalar optimizations. A problem with this approach is that incorrect assumptions about no-alias relationships between pointer pairs can optimize the program in such a way that it leads to crashes at runtime, or incorrect/undefined behavior of the program. More recently, Phulia et al. [49] exploited C's unspecified

order of evaluation to statically infer must-not-alias relationships missed by traditional analyses. These additional disambiguations enable compilers to perform more aggressive optimizations by proving that certain memory accesses cannot interfere. This is a user-annotated approach. Thus, this approach is prone to human error and may focus on code regions that have minimal impact on performance. ORAQL [33] is another optimistic approach that tries to estimate the gap between the current and ideal performance of HPC applications. Here, ideal performance is said to be achieved in the presence of highly precise no-alias information.

The approach proposed in ORAQL [33] begins optimistically, treating every may-alias pointer pair as no-alias, then compiles and executes the program, and compares the output with the expected program output. If the output matches, the optimistic assumption holds. If not (or the program results in an error), ORAQL iteratively tries to make the assumption less optimistic, using a bisection strategy, iteratively dividing the set of alias queries into two halves until all queries can be answered conclusively.

Consider four alias queries, q_1 , q_2 , q_3 , and q_4 . ORAQL begins optimistically and runs verification for all queries. On failure, it bisects into left ($\{q_1, q_2\}$) and right ($\{q_3, q_4\}$) chunks and tests each. If the left fails, it is split into q_1 and q_2 ; suppose q_1 passes while q_2 fails, so q_2 is marked pessimistic. The right chunk is handled similarly. This example requires seven compile-and-run iterations, leaving q_1 and q_3 optimistic and q_2 and q_4 pessimistic. This example also demonstrates ORAQL's worst case, which occurs when every query (or every alternate query) might overlap, forcing each query to be tested in isolation and requiring $2n - 1$ compile-and-run iterations, where n is the number of queries, potentially making it impractical for real-world benchmarks.

In this work, we propose HORIZON, a profiling-based tool to estimate the precision improvement upper bound of existing alias analysis implementations and its impact on the performance. We integrated HORIZON into a new LLVM-based tool. HORIZON annotates the program with the alias information (no-alias and must-alias) missed by existing implementations. HORIZON infers the annotations in an automated manner by instrumenting and executing the program. These annotations improve the precision and allow the compiler to explore missed optimization opportunities. Thus, we can analyze the impact of these additional optimizations on the performance of sequential and multi-threaded programs. Additionally, developers can use the inferred annotations to safely insert `restrict` keyword in cases where non-aliasing is guaranteed for given inputs but not inferred statically. We also expose cases of inefficient code generation caused by overly aggressive optimizations (Figure 6), helping compiler developers identify and fix optimization bugs.

Overall, ORAQL and HORIZON share the same goal but differ importantly. ORAQL uses a bisection strategy to separate pointer pairs that do not alias from the ones that may alias,

and is an optimistic or assumption-driven method, whereas HORIZON uses a profiling-based approach to infer must and no-alias relationships at runtime and thus identifies true alias relationships, which hold true only for the given inputs. Thus, unlike ORAQL, HORIZON can also handle must-alias queries. ORAQL relies on repeated compile-and-run iterations ($2n - 1$ iterations in the worst case), whereas HORIZON requires just two compile-and-run iterations, and the runtime depends only on the number of queries, irrespective of their result and order. The advantage of HORIZON w.r.t. ORAQL lies in producing input-sound annotations that are guaranteed to hold for the given inputs without relying on speculation. Furthermore, ORAQL focuses primarily on HPC applications (tested on a maximum of 50 KLOC), whereas HORIZON focuses on CPU-intensive workloads (tested on a maximum of 1304 KLOC).

This work provides answers to the following questions:

RQ1 For a program and given input test cases, what is the estimated upper bound on the precision improvement potential of alias analysis?

RQ2 How does the estimated range of precision improvement impact the performance of the program for the given input test cases?

RQ3 Which optimizations benefit the most from precise input-specific alias information for the program?

This paper makes the following contributions:

1. We present a tool to infer annotations holding precise alias information and to annotate the program with them.
2. We implemented our approach in an LLVM-integrated tool named HORIZON by adding new LLVM passes. We use those passes on some standard benchmarks to estimate the precision improvement potential (upper bound) of the default LLVM alias analysis.
3. We estimate the impact on the performance of the Polybench, CPU SPEC 2017, and SPLASH-2 benchmarks, when these benchmarks are annotated with the alias information missed earlier by static alias analysis.

Section 2 provides an overview of our approach. Section 3 presents the working of HORIZON in detail. Section 4 discusses the implementation details. Section 5 discusses the evaluation of HORIZON on Polybench, CPU SPEC 2017, and SPLASH-2 benchmarks. We discuss the prior work in Section 6 and conclude in Section 7.

2 Overview

Consider the code snippet in Figure 1a, taken from the `kernel_bicg` function of the `bicg` benchmark in the Polybench suite. This function implements the biconjugate gradient stabilized method for solving nonsymmetric linear systems. The code snippet contains a nested loop that updates the arrays s and q in each iteration. The arrays s , q , r , p , and A are passed as arguments to the function and do not have overlapping memory addresses for the default input test case.

```
static void kernel_bicg(int m, int n, double A[],
    double s[], double q[], double p[], double r[]) {
    for (i = 0; i < _PB_N; i++) {
        q[i] = SCALAR_VAL(0.0);
        for (j = 0; j < _PB_M; j++) {
            s[j] = s[j] + r[i] * A[i][j];
            q[i] = q[i] + A[i][j] * p[j];
        }
    }
}
```

(a) Bicg Polybench (kernel_bicg function).

```
static void kernel_bicg(...) {
    for (i = 0; i < _PB_N; i++) {
        q[i] = SCALAR_VAL(0.0);
        for (j = 0; j < _PB_M; j++) {
            s[j] = s[j] + r[i] * A[i][j];
            q[i] = q[i] + A[i][j] * p[j];
            Instrument(&s[j], &A[i][j]);
            Instrument(&s[j], &q[i]);
            Instrument(&s[j], &p[j]);
            Instrument(&s[j], &r[i]);
            ...
        }
    }
}
```

(b) Instrumented code for kernel_bicg function.

```
static void kernel_bicg(...) {
    for (i = 0; i < _PB_N; i++) {
        q[i] = SCALAR_VAL(0.0);
        for (j = 0; j < _PB_M; j++) {
            s[j] = s[j] + r[i] * A[i][j];
            q[i] = q[i] + A[i][j] * p[j];
            llvm.noalias.dynaa(&s[j], &q[i]);
            llvm.noalias.dynaa(&s[j], &r[i]);
            llvm.noalias.dynaa(&s[j], &A[i][j]);
            llvm.noalias.dynaa(&s[j], &p[i]);
            ...
        }
    }
}
```

(c) Annotated code for kernel_bicg function.

```
static void kernel_bicg(...) {
    for (i = 0; i < _PB_N; i++) {
        q[i] = SCALAR_VAL(0.0);
        double tmp = q[i];
        for (j = 0; j < _PB_M; j++) {
            s[j] = s[j] + r[i] * A[i][j];
            tmp = tmp + A[i][j] * p[j];
        }
        q[i] = tmp;
    }
}
```

(d) Optimized code for kernel_bicg function.

Figure 1. An example to discuss HORIZON approach.

Since the alias analysis performed is intra-procedural, it cannot determine whether arrays s and q overlap in memory. Most alias analysis implementations would adopt a conservative approach and assume a may-alias relationship for these arrays. Consequently, the compiler cannot optimize the code

further due to the lack of deterministic information about the function arguments.

We now discuss the approach followed by HORIZON using the code snippet in Figure 1a. HORIZON first profiles the application to infer alias relationships that cannot be determined by existing alias analysis implementations. It then annotates the program with the inferred alias relationships and recompiles it. These annotations may enable the compiler to exploit missed optimization opportunities.

Figure 1b shows the instrumented code by HORIZON to infer missing aliases. Instrumentation is added for different pairs of pointers present in the code snippet. For example, HORIZON instruments the pairs $(s[j], q[i])$, $(s[j], A[i][j])$, and $(q[i], A[i][j])$. The `Instrument(s[j], q[i])` call represents the instrumentation added for the pointer pair $(s[j], q[i])$. The implementation of `Instrument(P1, P2)` is shown in Figure 3. Executing this instrumented code generates annotations containing alias information missed by existing alias analyses.

Figure 1c shows the code snippet after incorporating the information collected from profiling. The `llvm.noalias.dynaa(&s[j], &q[i])` call represents an annotation added as metadata to the code. This annotation encodes the information that the j th element of array s do not alias with the i th element of array q . The description of `llvm.noalias.dynaa` is provided in Section 4.

Based on the no-alias relationship determined between arrays s and q , optimizations such as global value numbering (GVN) and loop invariant code motion (LICM) further improve the code, as shown in Figure 1d. GVN hoists the memory access to the i th element of array q out of the inner loop. LICM introduces a temporary variable tmp to store intermediate results and moves the store to memory location $q[i]$ outside the inner loop, applying the final value after the loop finishes. These optimizations reduce two memory accesses within the inner loop. The annotations improved the precision of alias analysis by 5%, resulting in a 53% performance improvement over the originally optimized version for the provided input test case.

HORIZON enables the compiler to exploit optimization opportunities that are otherwise missed, including code vectorization, loop-invariant code motion, load elimination, and store elimination, thereby realizing performance gains when precise alias information is available. In the next section, we describe HORIZON in detail.

3 HORIZON

3.1 Design Overview

HORIZON executes the program to infer alias information missed by static alias analysis and annotates the program with the obtained information. This allows the compiler to explore potential optimization opportunities on the basis of the new information. HORIZON works in four phases:

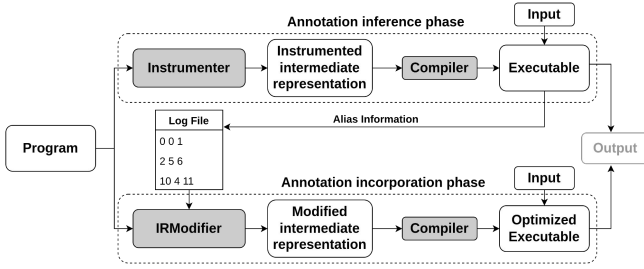


Figure 2. Architecture of HORIZON.

1. **Annotation Inference:** The input to the annotation inference phase is the optimized intermediate representation (IR) of the program, as shown in Figure 2. In this phase, HORIZON instruments the input IR to track alias relationships missed by the static analyzer. After instrumenting the code, the IR is compiled into an executable. HORIZON then executes the executable using default input test cases to generate a log file that contains the dynamically inferred alias relationships. The details of this phase are described in Section 3.2.
2. **Annotation Incorporation:** The program IR (optimized) and the log file generated by the annotation inference phase (Figure 2) are provided as input to this phase. In this phase, HORIZON annotates the IR with the alias information present in the log file. The annotated IR is then compiled into an executable. During the compilation, the optimization phases use the alias annotations present in the IR to generate efficient code. The details of this phase are described in Section 3.3.
3. **Precision Analysis:** The input to this phase is the native and annotated Intermediate Representations (IRs). HORIZON compiles both the IRs and analyzes the precision of the alias analysis implementation. The upper bound estimates on the precision improvement potential of LLVM’s default alias analysis implementation on some standard benchmarks are shown in Section 5.
4. **Performance Analysis:** The input to this phase is the native and optimized executables, both of which are generated in the annotation incorporation phase. HORIZON executes both executables on default input test cases and analyzes their performances. The performance analyses of some widely used benchmarks are described in Section 5.

3.2 Annotation Inference Phase

The goal of the annotation inference phase is to dynamically infer precise alias annotations for pointer pairs missed by the static alias analysis implementation. Towards this goal, HORIZON maintains a list, *tracking-list*, that contains all the pointers that may need dynamic tracking. It first adds the pointer operands of all loads, stores, atomic instructions, and memory intrinsic (e.g., `memcpy`) to the *tracking-list*. A pointer operand is either the address of an array element or the address of an array element’s internal field (if the array

```

struct MetaDataTy {
    unsigned long long offset:48,
    unsigned long long alias:1
};
Initialize() {
    for each pointer pair (p1, p2) {
        MetaData[p1, p2].offset = MAX_OFFSET;
        MetaData[p1, p2].alias = 1;
    }
}
Instrument(p1, p2) {
    offset = abs(p1 - p2);
    MetaData[p1, p2].alias &= (offset == 0);
    if (offset < MetaData[p1, p2].offset)
        MetaData[p1, p2].offset = offset;
}

```

Figure 3. Annotation inference logic. The minimum absolute relative offset between $p1$ and $p2$ is stored at runtime. The alias field in the metadata is set to zero if there is at least one instance when $p1$ and $p2$ are not equal.

is an array of structures). HORIZON then computes the base address of the array corresponding to each element in the *tracking-list*, by recursively tracking the pointer arithmetic and typecast operations. All the computed bases are also added to the *tracking-list*. The *tracking-list* also contains the pointer function arguments.

In compiler analysis, one pointer dominates another if every control-flow path reaching the second must pass through the first. Instrumentation can only be placed at program points dominated by both pointers. Therefore, HORIZON cannot track the pointer pairs where neither of the pointers dominates the other one. The queries made by the optimization passes for such pointer pairs are significantly fewer.

Instrumentation Logic. HORIZON infers annotations for each unordered pointer pair $\{p1, p2\}$, where $p1$ and $p2$ belong to the *tracking-list* and the static alias information is not available for $(p1, p2)$ pair. For every function, HORIZON allocates an array, `MetaData`, of $(N*N/2)$ elements, where N is the number of elements in the *tracking-list*. For each pointer pair, it assigns a unique index in the `MetaData` array.

Figure 3 shows the annotation inference logic. For each instrumented pointer pair $\{p1, p2\}$, HORIZON tracks the minimum absolute relative difference between $p1$ and $p2$. In addition to this, HORIZON also tracks the fact whether the pointer pairs always alias. This is required for the case when $p1$ and $p2$ both alias and do not alias (at different times) during the program execution (*may-alias*). At the entry of the main routine, the `Initialize` routine is called. It sets the `offset` and `alias` fields in the metadata to the maximum possible offset (`MAX_OFFSET = (1ULL << 48)`) and one, respectively, for each unordered pointer pair in the *tracking-list*. Before exiting the program, HORIZON logs the metadata to a file that is used by the annotation incorporation phase (Section 3.3). If the offset is `MAX_OFFSET` in the log file, the corresponding code region was never executed during profiling.


```

checkAlias(void *p1, void *p2, size_t p1_size, size_t p2_size);

newCheckAlias(void *p1, void *p2,
               size_t p1_size, size_t p2_size) {
    Ret = checkAlias(p1, p2, p1_size, p2_size);
    if (Ret == DONT_KNOW)
        for each metadata intrinsic (X, Y, offset)
            if ((p1 == X || p2 == X) && (p1 == Y || p2 == Y))
                if (offset >= max(p1_size, p2_size)) return NOT_ALIAS;
                else return PARTIAL_ALIAS;
    return Ret;
}

```

Figure 4. The original `checkAlias` interface takes a pointer pair and their respective sizes as inputs. The `newCheckAlias` routine, if required, also walks all the metadata intrinsics to check if an intrinsic contains the input pointer pair and an offset that is greater than the sizes of both the pointers.

The log file entry corresponding to pointer pair $\{p1, p2\}$ is interpreted as follows:

1. If `offset = 0` and `alias = 1`, then $p1$ and $p2$ are aliases.
2. If (`offset = 0` and `alias = 0`) or `offset = MAX_OFFSET`, then $p1$ and $p2$ may or may not alias.
3. If `offset > 0`, then $p1$ and $p2$ either partially alias or do not alias.

3.3 Annotation Incorporation Phase

During the annotation incorporation phase, HORIZON annotates the IR with the alias information present in the log file generated during the inference phase (Section 3.2). If an unordered pair of pointers $\{p1, p2\}$ is inferred as an alias (*must-alias*), and $p1$ dominates $p2$, HORIZON replaces all uses of $p2$ with $p1$. If the types of $p1$ and $p2$ are different, $p2$ is replaced after typecasting $p1$ to $p2$'s type. After this step, the compiler can statically infer that all usages of $p2$ and $p1$ are pointing to the same address. However, *no-alias* cases are challenging since the IR cannot replace all uses of one pointer with another. Instead, HORIZON encodes inferred *no-alias* pairs using LLVM `alias.scope` and `noalias` metadata, which `ScopedNoAliasAA` interprets to enable optimizations without changing program semantics. We identify the pointer pairs that have non-overlapping memory addresses on the basis of the size information and the absolute minimum difference between pointer pairs. The `alias.scope` and `noalias` metadata are attached to such non-overlapping memory accesses. Currently, the `ScopedNoAliasAA` pass of LLVM provides metadata support for the limited set of instructions (i.e. memory access instructions). Thus, to incorporate the information about *no-alias* pointer pairs other than the memory access instructions, we also modify the LLVM's backend.

Consider the `checkAlias` function in Figure 4. `checkAlias` is the original alias analyzer interface for alias queries. The `checkAlias` function takes two pointers $p1$ and $p2$, and their sizes `p1_size` and `p2_size` as input. The `checkAlias` function reports a pointer pair

to be *no-alias*, if it can statically infer that the memory regions " $p1, p1+p1_size$ " and " $p2, p2+p2_size$ " do not overlap. To answer these queries, HORIZON annotates the source code with metadata intrinsics. A metadata intrinsic is ignored by the compiler during code generation but retained during intermediate program transformations. Our metadata intrinsic contains a pair of pointers and the minimum absolute relative offset observed during the inference phase (Section 3.2).

We have modified the `checkAlias` interface to `newCheckAlias` (see Figure 4). `newCheckAlias` additionally iterates the metadata intrinsics to find an answer if the `checkAlias` does not have a definite answer. The prototype of `newCheckAlias` is the same as `checkAlias`. If `newCheckAlias` finds metadata intrinsic that contains $p1$ and $p2$ and the offset is greater than or equal to the maximum of `p1_size` and `p2_size`, it reports them as *no-alias*. Note that the offset is the minimum absolute relative offset observed during the annotation inference phase. If `offset >= max(p1_size, p2_size)`, then the memory regions " $p1, p1+p1_size$ " and " $p2, p2+p2_size$ " do not overlap. However, this is not the best answer we can give. For example, consider a case when `p1_size` is less than `p2_size`. At runtime, the minimum relative offsets are `p1_size` if $p1 < p2$ and `p2_size` if $p2 < p1$. In this case, if the offset is greater than or equal to `p1_size`, we can safely conclude that $p1$ and $p2$ are *no-alias*. To support this, we need to track two offsets during the annotation inference phase. The first one is the minimum relative offset if $p1 < p2$, and the second one is the minimum relative offset if $p2 < p1$. We have not implemented this feature. To estimate the loose upper bound of the precision loss due to this restriction, we counted the number of queries where the runtime minimum offset lies between the size of input pointer pairs. The maximum number of such queries is 88 for the `500.perlbench_r` benchmark. Notice that even if two offsets are logged during the annotation inference phase, some queries may remain unanswered.

To estimate the potential precision improvement that could be achieved with a tighter bound for offset, we counted the percentage increase in the inferred pointer pairs with the condition `offset >= min(p1_size, p2_size)` compared to the original condition `offset >= max(p1_size, p2_size)`. Polybench showed no improvement. In CPU SPEC 2017, six benchmarks exhibited 0–2% improvement, while `544.nab_r` and `502.gcc_r` showed higher gains of 4.05% and 2.8%, respectively. SPLASH-2 exhibited mixed results, seven benchmarks achieved 0–2% improvement, two fell in the 2–3% range, and `ocean_ncp` achieved a 10% improvement.

4 Implementation

We implemented HORIZON as part of the LLVM infrastructure (v10.0.0). Our initial motivation for developing HORIZON

was to establish a foundation with the essential features to support our broader research in compiler optimization. Over time, the tool has matured and demonstrated its value, and we believe it can now benefit other researchers, despite relying on an older LLVM version. We plan to extend support to the latest LLVM version in the near future to ensure broader compatibility and facilitate wider adoption.

The default implementation of alias analysis in LLVM involves four passes: *basic alias analysis*, *type-based alias analysis*, *scoped no-alias analysis*, and *globals mod/ref alias analysis*. For an alias query, these analyses are performed sequentially until a definitive answer is reached. In LLVM, the alias analysis results are used for optimizations such as global value numbering (GVN), loop invariant code motion (LICM), dead store elimination (DSE), loop vectorization (LV), and superword-level parallelism vectorization (SLP).

HORIZON works on an optimized intermediate representation (IR) of the program, as described in section 2. The input to the annotation inference phase (Section 3.2) and annotation incorporation phase (Section 3.3) is an optimized LLVM IR compiled with the “O3” optimization level. We disable SLP vectorization, loop vectorization, and loop unrolling while generating the input IR. We found that the vectorization and loop unrolling passes cannot further optimize an already vectorized/unrolled loop with the extra alias information generated using the annotation inference phase. However, if loop unrolling and vectorization are first performed on an IR after the additional alias information is incorporated, LLVM can vectorize and unroll better than it does without the additional alias information. So, it is better to disable these optimizations during the initial optimization and only enable them later, when annotations have been incorporated.

We have added two new passes to LLVM. The first pass, known as “dynaa”, performs the annotation inference phase. The second pass, known as “get-dynaa”, performs the annotation incorporation phase. HORIZON also annotates the source code with alias information, as described in section 2. The annotation inference pass selectively instruments the functions that contribute significantly (>95%) to the total execution time of the benchmark. The per-function execution time is obtained using Gprof [24] tool. The annotation incorporation phase incorporates the information (generated during the annotation inference) from the log file phase into the IR. The incorporation of *must-alias* information is simple and has been described in Section 3.3. The remaining information present in the log file is added using an LLVM intrinsic described in Section 4. The resulting IR is further provided to the other optimization passes to leverage the information collected by HORIZON. Whenever the default implementation of alias analysis fails to provide a definitive answer for a pointer pair, the result is checked in the newly added information by parsing through the metadata intrinsics (newAliasCheck routine in Figure 4). In this case, the result can be either *no-alias* or *may-alias*.

Adding noalias-intrinsic nodes. LLVM intrinsics can preserve metadata for the program throughout the transformation passes. We have added a new intrinsic named “llvm.noalias.dynaa” to the LLVM backend. This intrinsic takes three arguments. The first two arguments are pointers wrapped in the form of “LLVM’s metadata nodes,” and the third argument is a 64-bit integer. This type of intrinsic can also be referred to as a debug intrinsic since the arguments passed to this intrinsic hold the metadata for the program. Thus, the program needs to be compiled in debug mode (-g option). A sample usage of the intrinsic is this:

```
call void @llvm.noalias.dynaa(metadata DATA_TYPE
%1, metadata DATA_TYPE %2, metadata i64 c)
```

where metadata represents LLVM’s metadata node, DATA_TYPE represents the pointer type, i64 represents a 64-bit integer, %1 and %2 represent pointers, and c represents a constant integer value. We refer to this intrinsic using noalias-intrinsic.

Preserving Alias Annotations with IR Transformations. The challenge in the annotation incorporation phase is to preserve and update the pointers for which annotations are added by the LLVM transformation passes. The *must-alias* information is incorporated into the IR by replacing all uses of a pointer with its alias (another pointer), as described in section 3.3. This ensures the perseverance of collected *must-alias* information throughout the LLVM transformation passes. The remaining information in log file is incorporated into IR using the noalias-intrinsic. We leverage the features of intrinsic and metadata nodes present in LLVM to preserve and update the annotations added by HORIZON. If a pointer with annotation information is deleted from IR, the compiler also invalidates the corresponding metadata node/intrinsic. If a pointer is updated, the compiler updates the corresponding metadata nodes/intrinsics. We also add an extra noalias-intrinsic if an operand X of the original noalias-intrinsic is obtained from a typecast operation. The operand Y (corresponding to X) for the extra node is obtained after removing all typecast operations on X. This is needed to handle the case when X is deleted from IR, but Y is still present. To find a noalias-intrinsic corresponding to a pointer pair {P1, P2}, HORIZON looks for a noalias-intrinsic that contains an annotation for P1 and P2, or statically known aliases (e.g., typecast) of P1 and P2 as operands. This is needed for newly added definitions to IR when noalias-intrinsic is available for the new node’s statically known aliases instead of the pointer pair.

Importance of Input Test Cases. In our study, we used HORIZON to infer alias annotations for the default input test cases of the chosen benchmarks. The quality of the inferred annotations depends on the test cases provided while executing the instrumented program. The more general the test cases provided, the more reliable (always true) the inferred annotations. We used standard benchmarks in the evaluation process. These benchmarks come with certain default test

cases whose quality can be trusted. For the benchmarks that run on multiple test cases, HORIZON generates a log file that stores cumulative annotations for the entire set of test cases. These annotations are inferred by executing the benchmark on all test cases. This ensures that the benchmark does not crash or result in undefined behavior for any of the test cases.

5 Evaluation

We evaluated HORIZON on the default implementation of alias analysis in LLVM using 30 Polybench [8] benchmarks, eight C benchmarks which are available as a part of Intrate and FPrate suites in CPU SPEC 2017, and 10 SPLASH-2 [1] benchmarks. For Polybench benchmarks, the extra-large input set has been used for the evaluation. For the CPU SPEC 2017 benchmarks, we have used the reference input set. Table 1 describes the input set used for three benchmarks of SPLASH-2 benchmark suite and for the remaining seven benchmarks we used the native input set mentioned in [2]. In order to understand the impact on the performance, benchmarks need to have reasonably high execution times. Therefore, we have used custom inputs to ensure reasonably high execution times for three benchmarks mentioned in Table 1. For simplicity, we refer ocean (contiguous_partitions) as ocean_cp, ocean (non_contiguous_partitions) as ocean_ncp, lu (contiguous_blocks) as lu_cb and lu (non_contiguous_blocks) as lu_ncb. The experiments were performed on 2.10GHz Intel(R) Xeon(R) Silver 4116 CPU 12 core machine with an x86_64 architecture and 32 GB primary memory which uses a 64-bit Ubuntu 18.04.1 operating system. We had disabled the hyper-threading during our experiments. For performance numbers, we used the minimum runtime out of the five runs for each benchmark. The source code and other artifacts are available at URLs <https://doi.org/10.5281/zenodo.17939867> and https://github.com/khushboochitre/artifact_horizon.git.

Table 1. Input set description for SPLASH-2 benchmarks.

Benchmark	Input set
ocean_ncp	4098 x 4098 Grid, Timestep = 28800, Distance between grid points = 20000, Error tolerance = 1e-07
radiosity	Largeroom, BF refinement = 0.00025
raytrace	Environment file = car.env, Antialiasing with subpixels = 32768

Since ORAQL and HORIZON largely have the same goal, we tried to evaluate and compare the performance of both tools on the selected benchmarks. However, we were unable to gather results for ORAQL because the artifacts for the tool were only partially available, leading to segmentation faults for the selected benchmarks (CPU SPEC 2017).

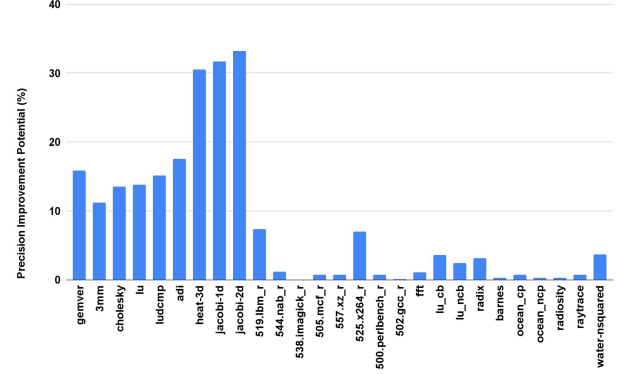


Figure 5. Precision improvement potential (%) for Polybench, CPU SPEC 2017 and SPLASH-2 benchmarks.

5.1 Precision Improvement Potential

RQ1 For a program and given input test cases, what is the estimated upper bound on the precision improvement potential of alias analysis?

The precision of an alias analysis implementation is defined as the degree to which it answers pointer pairs correctly. Out of all the information obtained from alias analysis, the information pertaining to no-alias and must-alias pointer pairs is the most useful. Optimizations rely on this information to transform the program to a more efficient semantically equivalent version. Less precise implementations of alias analysis tend to be more conservative, and therefore, less likely to correctly classify a must-alias or no-alias pointer pair. Thus, if a specific implementation classifies more pointer pairs as either must-alias or no-alias for a particular program, it is likely to be more precise.

The graph shown in Figure 5 plots precision improvement potential which is computed as the percentage increase in the number of no-alias and must-alias pointer pairs when the Polybench, CPU SPEC 2017, and SPLASH-2 benchmarks are compiled with and without the program annotations. We use the LLVM’s “aa-eval” pass to generate this information. For Polybench, the graph depicts benchmarks for which the precision improvement potential is high. There are nine benchmarks for which the potential is greater than 10%. Out of these nine benchmarks, three benchmarks report an improvement potential of more than 30%. These results indicate that there is a substantial potential to improve the precision of the alias analysis implementation for the default input test cases.

We further observe that the current implementation of static alias analysis in LLVM misses pointer pairs involving:

- Inter-procedural information such as function arguments or return values of functions and,
- Information depending on user input (dynamic information) or conditional statements.

Table 2. Performance impact (%) for Polybench and CPU SPEC 2017 benchmarks (AO = All Optimizations enabled, MEO = Most Effective Optimizations enabled).

Benchmark	AO (%)	MEO (%)
gemver	-1.02	0.2
gesummv	46.59	47.38
bicg	53.16	53.32
mvt	-0.37	0.36
adi	-7.67	0.33
jacobi-1d	20.47	21.11
519.lbm_r	1.15	1.15
544.nab_r	0.77	1.02
538.imagick_r	0.94	1.17
505.mcf_r	2.06	3.65
557.xz_r	1.64	2.1
525.x264_r	-1	11.56
500.perlbench_r	0.51	1.02
502.gcc_r	3.48	3.82

The information related to function arguments and return values of functions can be determined statically by performing inter-procedural analysis. However, most alias analysis implementations avoid performing inter-procedural analysis due to its expensive nature. We observed that most of the pointer pairs missed for benchmarks such as jacobi-2d, heat-3d, jacobi-1d, adi, 3mm, cholesky, lu, ludcmp and gemver involve function arguments. These pointer pairs were missed due to the absence of inter-procedural alias information, as shown in Figure 5. HORIZON was able to provide precise answers for such pointer pairs. Therefore, the precision improvement scope for these benchmarks is significantly higher than the other benchmarks.

A large majority of benchmarks (39 of 48) demonstrate a precision improvement scope between 0% and 10%. Remaining nine benchmarks show a very high (greater than 10%) precision improvement potential. These estimates depend on the input test cases during the annotation inference phase.

5.2 Performance Improvement Potential

RQ2 How does the estimated range of precision improvement impact the performance of the program for the given input test cases?

We estimated the impact on the performance of the Polybench and CPU SPEC 2017 benchmarks by incorporating the program annotations inferred by HORIZON for the provided input test cases. We did this by computing the percentage change in the execution time of each benchmark compiled with and without annotations. Since it was discovered that certain optimizations can actually harm performance, we also tried calculating the percentage improvement in execution time for each benchmark while keeping only the most effective combination of optimizations enabled. Table 2

```
static void kernel_adi(int tsteps, int n,
double u[], double v[], double p[], double q[]) {
    for (j=1; j<_PB_N-1; j++) {
        p[i][j] = -c / (a*p[i][j-1]+b);
        q[i][j] = (-d*u[j][i-1]+(SCALAR_VAL(1.0)+ SCALAR_VAL(2.0)
        *d)*u[j][i] - f*u[j][i+1] - a*q[i][j-1])/(a*p[i][j-1]+b);
    }
}
```

Figure 6. Example (adi) with negative performance impact of -8%.

```
static void kernel_gesummv(int n, double alpha,
double beta, double A[], double B[], double tmp[],
double x[], double y[]) {
    for (i = 0; i < _PB_N; i++) {
        tmp[i] = SCALAR_VAL(0.0);
        y[i] = SCALAR_VAL(0.0);
        for (j = 0; j < _PB_N; j++) {
            tmp[i] = A[i][j] * x[j] + tmp[i];
            y[i] = B[i][j] * x[j] + y[i];
        }
        y[i] = alpha * tmp[i] + beta * y[i];
    }
}

static void kernel_jacobi_1d(int steps,
int n, double A[], double B[]) {
    for (i = 1; i < _PB_N - 1; i++)
        B[i] = 0.33333 * (A[i-1] + A[i] + A[i + 1]);
    for (i = 1; i < _PB_N - 1; i++)
        A[i] = 0.33333 * (B[i-1] + B[i] + B[i + 1]);
}
```

Figure 7. Examples (gesummv and jacobi-1d) with positive performance impact of 47% and 21% respectively.

shows these results for the CPU SPEC 2017 benchmarks and for the Polybench benchmarks that result in a significantly high performance impact (positive or negative).

By examining the IR and statistics in Table 2, we make the following key observations:

- In bicg (Figure 1a), the no-alias relationship inferred by HORIZON between arrays s , r , p , q , and A enables GVN and LICM. GVN hoists the load of $r[i]$ outside the inner loop since $r[i]$ is never modified inside the loop. LICM moves the store to $q[i]$ to the end block of the inner loop, storing the intermediate results for the operation into a temporary variable. The final result is stored into $q[i]$ when the inner loop has finished executing. These optimizations further unroll the loop body by a factor of two.
- In gesummv (Figure 7), the no-alias relationship inferred by HORIZON between arrays tmp , A , B , x and y enables GVN and LICM. LICM moves the store to $tmp[i]$ and $y[i]$ out of the inner loop by storing the intermediate results in temporary variables. The final result for $tmp[i]$ is stored right after the inner loop finishes execution. GVN removes the load for $tmp[i]$ as the value for $tmp[i]$ can be obtained from the temporary variable used earlier to store its value. These optimizations further enable SLP vectorization for the operations being performed in the inner loop.

- In `jacobi-1d` (Figure 7), the no-alias relationship inferred by HORIZON between $A[i-1]$, $A[i]$, $A[i+1]$ and $B[i]$ allows GVN to remove the load for $A[i-1]$ and $A[i]$ as these values are respectively equivalent to $A[i]$ and $A[i+1]$ from the previous iteration. This reduces the inner-loop loads from three to one. The same optimization applies to the second inner loop loading $B[i]$, $B[i-1]$, and $B[i+1]$. It also aids loop vectorization by avoiding redundant loads of $A[i-1]$, $A[i]$, $B[i-1]$, and $B[i]$.
- In `adi` (Figure 6), these instructions were vectorized,
 $\%v3 = \%v1 / \%v2; \%v4 = \text{const} / \%v2;$
 The corresponding vectorized code had two insertelements to create the dividend, one insertelement and a shufflevector to create the divisor, a vectorized division operation followed by two extractelements to perform different scalar operations on quotients. The vector result of division is reused in the next iteration of a loop (i.e., the result is an operand of a PHI). Two extractelement to perform different scalar operations on previous quotients. The unvectorized code is simpler and more efficient. Table 2 shows that the performance of `adi` improves by 0.33% when SLP vectorization is disabled during compilation.
- In `525.x264_r`, `pixel_hadamard_ac` function degrades the performance of the benchmark. The no-alias information inferred by HORIZON enables SLP vectorization in `pixel_hadamard_ac` due to the removal of one load by GVN. This ends up adding extra instructions to the IR, increasing the overall cost of the function. Disallowing GVN while compiling the program with annotations improves the overall performance by 12%.

We observed that some benchmarks showed performance differences when compiled with all optimizations versus a subset of them. Interestingly, disabling certain optimizations can improve performance compared to enabling all. Table 4 lists all the optimizations that do not degrade performance for these benchmarks. For most of the Polybench and CPU SPEC 2017 benchmarks, the impact due to increased alias analysis precision on the performance of the benchmarks lies between 0% to 10%. The occasional performance degradation can happen when aggressive optimizations triggers wrong optimizations or an increase in code size, degrading performance. In such cases, disabling specific optimizations can improve benchmark performance.

Performance Improvement Potential of Multi-threaded Benchmarks. To assess the impact on the performance of multi-threaded programs using the annotations inferred by HORIZON, we use the multi-threaded SPLASH-2 benchmarks. We execute the benchmarks for different thread counts (1, 2, 4, 8, and 12), compiled with and without annotations. Benchmarks `fft`, `radix`, `ocean_cp`, and `ocean_ncp` execute only when the thread count is a power of 2. These benchmarks have properties that make the performance dependent on the number of threads [62].

Table 3 shows the performance impact (%) with different thread counts for each benchmark when compiled with all optimizations and the most effective combination of them. For multithread applications, compiler optimizations can further impact the behavior of cache contention. We measure the impact on the cache contention using the number of HITM (loads that hit in a modified cache) events [3] observed by the `perf` [35] tool. Reduction in HITM events is directly proportional to the reduction in cache contention.

We observe that `raytrace` shows a significant improvement compared to the other benchmarks when compiled using annotations. The main reason for this is a significant reduction in loads (22-30%), stores (23-25%), and HITM events (11-32%) for different numbers of threads. The reduction in HITM events (32%), loads (30%) and stores (25%) is the highest with two threads while lowest (11%) for four threads.

For `radiosity`, the improvement is due to the reduction in loads (1-5%), stores (9-15%), and HITM events (-16-38%). The best performance improvement is achieved with one thread, and then it consistently decreases with the growing number of threads. This happens because as the number of threads increases, the reduction of HITM events (maximum for two threads, at 31.72%) decreases. For 12 threads, the number of HITM events is increased by 16%.

For `water-nsquared`, precise alias information leads to an approximate reduction of 50% in both loads and stores, irrespective of the number of threads, resulting in a good performance improvement. The performance gains increase with multiple (greater than one) threads due to a 73–98% reduction in HITM events.

For `ocean_cp`, the number of loads decreases by approximately 10% for all threads. However, the number of HITM events increases by 18% for two threads and 4% for eight threads. For four threads, the number of HITM events decreases by 0.3%. For this benchmark, we don't observe a significant improvement for multiple threads.

We observe that the benchmarks in Table 2, except those discussed above, show performance differences when compiled with all optimizations versus a subset of them. Table 4 lists all the optimizations that don't cause any performance degradation for these benchmarks. The most effective optimization combination may change as the thread count increases since more threads alter cache behavior and, in turn, optimization effectiveness.

Adding the 'restrict' keyword on function arguments based on HORIZON's inferred annotations. We analyzed the impact of adding "restrict" to the function arguments based on HORIZON's annotations. To safely insert "restrict", we identified the function arguments that were no-alias with all other pointers (not identified precisely by static analysis) based on the inferred annotations. At the IR level, we realize "restrict" by attaching LLVM's `NoAlias` attribute to the corresponding function arguments. It is worth noting that for all

Table 3. Performance impact in percentages (AO = All Optimizations enabled, MEO = Most Effective Optimizations enabled.)

	#Threads = 1		#Threads = 2		#Threads = 4		#Threads = 8		#Threads = 12	
Benchmark	AO	MEO	AO	MEO	AO	MEO	AO	MEO	AO	MEO
fft	-0.13	1.56	-0.21	0.45	-0.13	0.25	0.05	0.05	–	–
lu_cb	9.87	9.9	10.43	10.54	10.62	10.71	9.42	9.42	6.3	6.3
lu_ncb	8.48	8.75	8.43	8.85	7.18	7.44	5.11	5.58	5.78	6.18
radix	-0.06	0.06	0.44	0.72	0	0.3	0.18	0.69	–	–
barnes	5.83	6.09	5.13	5.13	7.64	7.64	4.87	5.53	6.23	6.23
ocean_cp	5.16	5.26	0.02	0.09	0.1	0.14	-0.02	0.02	–	–
ocean_ncp	0.42	0.69	0.92	1.17	0.58	0.58	-0.24	0.08	–	–
radiosity	16.97	17.98	15.99	17.36	14.93	15.68	7.51	10	7.25	8.7
raytrace	34.35	34.35	41.77	41.77	16.35	17.39	19.95	20.23	23.46	23.9
water-nsquared	3.44	4.41	11.74	12.73	9.94	10.61	11.14	11.59	9.73	10.57

experiments, the annotations were incorporated after function inlining, reducing the number of function arguments available.

For all Polybench benchmarks, inlining reduced the code to a single remaining function argument, limiting the scope of meaningful evaluation. For four of the eight CPU SPEC 2017 benchmarks, the performance improvement was in the 0-1% range, with a maximum of 0.92% for 500.perlbench_r. The remaining four benchmarks exhibited performance degradation, with a maximum of 1.12% for 557.xz_r. For the multi-threaded SPLASH-2 benchmarks, only two out of ten benchmarks reported performance degradations with a maximum of ~1.3% for fft (1 thread). Seven of the remaining eight benchmarks had performance improvements in the 0-10% range, with only radiosity showing an improvement of ~18.7% for 1 thread.

In the future, we plan to investigate incorporating annotations before function inlining. This would preserve a larger set of function arguments, enabling a more comprehensive evaluation of opportunities to insert “restrict” keyword.

5.3 Optimization Opportunities

RQ3 Which optimizations benefit the most from precise input-specific alias information for the program?

Table 4 lists the optimizations that contribute to achieving the best performance for each benchmark. We obtained this data by executing the benchmarks with different set of optimizations enabled. Among the optimizations, DSE and LICM appear most frequently, i.e., 42 and 36 out of the total 47 benchmarks respectively, whereas LV and SLP appear least frequently (24/47). All optimizations play a role in optimizing the four benchmarks for which we observed around 10% or more performance improvement (Table 2). In particular, DSE appears in all combinations for those benchmarks, whereas SLP appears only in one case. GVN, LV, and LICM each appear in three out of those four combinations. This trend continues across all benchmarks for which we observe that on average about four optimizations benefit from the

Table 4. Most Effective Optimizations.

Benchmarks	Combination
2mm, nussinov	LICM
durbin, fdtd-2d, 500.perlbench_r	DSE
lu_ncb	GVN, DSE
gemm, radix	LICM, LV
ludcmp, heat-3d, ocean_cp	LICM, DSE
544.nab_r	LV, DSE
mvt	GVN, LV, SLP
gemver, gesummv, jacobi-2d, 502.gcc_r	GVN, LV, DSE
water-nsquared	GVN, SLP, DSE
3mm, bigc, adi, 538.imagick_r, 505.mcf_r	GVN, LICM, DSE
557.xz_r	LICM, LV, SLP
correlation, syr2k, trisolv, seidel-2d, fft	LICM, SLP, DSE
atax, jacobi-1d, lu_cb	GVN, LICM, LV, DSE
trmm, doitgen, 519.lbm_r, ocean_ncp	GVN, LICM, SLP, DSE
barnes	GVN, LV, SLP, DSE
covariance, gramschmidt, floyd-warshall, 525.x264_r, radiosity	LICM, LV, SLP, DSE
symm, syrkc, cholesky, lu, deriche, raytrace	GVN, LICM, LV, SLP, DSE

availability of precise alias information. However, picking the right set of optimizations is tricky for multiple threads. A set of optimizations that works well for one thread might not be the best option for multiple threads, when cache contention also comes in the picture. For example, in raytrace, the set of optimizations that work for one and two threads is not the best option for other threads. The improvement for one and two threads is better when all optimizations are enabled (Table 3) whereas the other threads perform better when the optimization LICM is disabled.

5.4 Overhead of Annotation Inference Phase

The annotation inference phase infers the annotations by instrumenting the program as described in Section 3.2. The added instrumentation significantly affects the execution time of the program. The overhead depends on the number of pointer pairs instrumented. We observed that the overhead of the annotation inference phase has a positive correlation with the number of pointer pairs instrumented. The overhead lies in the range of 1x (seidel-2d) to 126x

(1u_cb) with an average overhead of 25x (with respect to the original execution time). 31 of the 48 tested benchmarks have an overhead of less than 25x. Even though the overhead is high, this is a one-time overhead that is incurred before the program is optimized and used in production. To reduce the overhead, in the future, we can selectively instrument only pointer pairs queried by the optimizations, using an additional profiling phase to identify those pairs and instrument them during annotation inference.

6 Related work

Alias analysis has been an active area of research for many years, and different aspects of alias analysis have been explored in the past. [30] nicely summarizes the possible research aspects in the field of alias analysis. The proposed alias analysis algorithms vary in the degree of precision and efficiency. In most cases, efficiency is prioritized over precision. However, precision plays an important role in recognizing optimization opportunities. Attempts have been made in the past to perform inter-procedural alias analysis, which is more precise than intra-procedural alias analysis.

Andersen’s inclusion-based analysis [11] and Steensgaard’s unification-based approach [53] became foundational techniques for interprocedural alias analysis, with Andersen’s method being precise but cubic in time and Steensgaard’s linear in time but less precise. Several extensions have been proposed to enhance the precision of Steensgaard’s method [20, 51] and to reduce the time complexity of Andersen’s [25, 26, 47, 48, 58]. Despite the precision improvements, the gains from these approaches were limited due to the absence of context sensitivity.

Building on this foundation, many inter-procedural and context-sensitive approaches have been proposed, such as, call-strings approach [22, 36, 42, 65], cloning-based approach [37, 57, 60] and summary-based approaches [16, 18, 31, 43, 44, 56, 61]. Recent approaches, such as SVF [55] and FSAM [54] (for multi-threaded programs), achieve context-sensitivity using sparse value flow graphs and summarization techniques. These offer superior precision but suffer from scalability limitations due to path explosion and memory requirements, in recursive or large modular programs.

Another class of approaches aims to further improve precision by increasing the number of no-alias responses using optimistic assumptions [17, 33, 49] or runtime pointer disambiguation [5, 10, 14, 15, 41, 45, 52]. These additional responses enable compilers to perform more aggressive optimizations by proving that certain memory accesses cannot interfere.

A related research direction explores speculative SSA forms, which extend the SSA form to incorporate speculative alias information [9, 13, 19, 23, 32, 38]. This information enables the compiler to aggressively perform speculative optimizations. Such techniques were highly effective on older hardware with high memory latency, but their benefits diminish on modern processors with larger caches and

sophisticated prefetching. They are most effective when recovery is rare and profiling indicates high confidence in alias assumptions. Some prior work [40, 63] employed instrumentation and dynamic analysis to detect bugs in alias analysis implementations, inspiring some of HORIZON’s ideas.

Weber et al. [59] argue against improving the precision of alias analysis by suggesting that only about 3% of alias information leads to beneficial optimizations. While there is merit to the idea, as indicated by some of our experiments (Section 5.3), our results indicate an overall performance benefit for most real-world benchmarks.

7 Conclusion

Compiler developers often compromise with alias analysis precision to achieve scalability. We presented HORIZON that estimates the upper bound on the precision improvement potential of alias analysis implementations. HORIZON infers the annotations of alias information automatically by executing the program on a given input test case and incorporate it into the program. We integrated HORIZON with LLVM and evaluated it for the default LLVM alias analysis. We found that improving the precision allowed the compiler to explore more optimization opportunities resulting in better performance for the given input test cases for some of the programs. The observed upper bound for precision improvement of LLVM’s default alias analysis implementation was found to be in the range of 0% to 33% for the default input test cases provided to the benchmarks. The performance impact lies in the range of 0% and 53% for the provided input test case, indicating that some benchmarks can benefit substantially.

Acknowledgments

We would like to thank the anonymous reviewers for their valuable feedback, that helped improve the presentation of the paper. The first author would like to acknowledge TCS Foundation for supporting her research through the TCS Research Scholar Program. The first author would also like to thank Qualcomm India for providing travel assistance and encouragement.

Data Availability Statement

The source code repository [6, 7] of HORIZON is available at URLs <https://doi.org/10.5281/zenodo.17999887> and https://github.com/khushboochitre/artifact_horizon.git.

References

- [1] 2007 (accessed Sep 6, 2025). Splash 2 benchmark suite. <http://web.archive.org/web/20070704172333/http://www-flash.stanford.edu/apps/SPLASH/splash2.tar.gz>
- [2] 2011. A Memo on Exploration of SPLASH-2 Input Sets.
- [3] 2017 (accessed Sep 6, 2025). Intel 64 and IA32 Architectures Performance Monitoring Events. <https://software.intel.com/content/>

- www.us/en/develop/download/intel-64-and-ia32-architectures-performance-monitoring-events.html.
- [4] 2021 (accessed Feb 26, 2021). LLVM Alias analysis Infrastructure. <https://llvm.org/docs/AliasAnalysis.html>
 - [5] 2025 (accessed Apr 12, 2025). Runtime Checks of Pointers. <https://llvm.org/docs/Vectorizers.html#runtime-checks-of-pointers>
 - [6] 2025 (accessed Dec 20, 2025). Horizon Artifact DOI. <https://doi.org/10.5281/zenodo.17999887>
 - [7] 2025 (accessed Dec 20, 2025). Horizon Artifact Github Repository. https://github.com/khushboochitre/artifact_horizon.git
 - [8] 2025 (accessed Sep 6, 2025). Polybench benchmark suite. <https://sourceforge.net/projects/polybench/>
 - [9] Wonsun Ahn, Yuelu Duan, and Josep Torrellas. 2013. DeAliaser: Alias Speculation Using Atomic Region Support. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems* (Houston, Texas, USA) (ASPLOS '13). Association for Computing Machinery, New York, NY, USA, 167–180. doi:10.1145/2451116.2451136
 - [10] Péricles Alves, Fabian Gruber, Johannes Doerfert, Alexandros Lamprineas, Tobias Grosser, Fabrice Rastello, and Fernando Magno Quintão Pereira. 2015. Runtime Pointer Disambiguation. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications* (Pittsburgh, PA, USA) (OOPSLA 2015). Association for Computing Machinery, New York, NY, USA, 589–606. doi:10.1145/2814270.2814285
 - [11] Lars Ole Andersen and Peter Lee. 2005. Program Analysis and Specialization for the C Programming Language. <https://api.semanticscholar.org/CorpusID:20876553>
 - [12] Michael G. Burke, Paul R. Carini, Jong-Deok Choi, and Michael Hind. 1994. Flow-Insensitive Interprocedural Alias Analysis in the Presence of Pointers. In *Proceedings of the 7th International Workshop on Languages and Compilers for Parallel Computing (LCPC '94)*. Springer-Verlag, Berlin, Heidelberg, 234–250.
 - [13] Tong Chen, Jin Lin, Xiaoru Dai, Wei-Chung Hsu, and Pen-Chung Yew. 2004. Data dependence profiling for speculative optimizations. In *International Conference on Compiler Construction*. Springer, 57–72. <https://api.semanticscholar.org/CorpusID:29714270>
 - [14] Khushboo Chitre, Piyus Kedia, and Rahul Purandare. 2022. The road not taken: exploring alias analysis based optimizations missed by the compiler. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 153 (Oct. 2022), 25 pages. doi:10.1145/3563316
 - [15] Khushboo Chitre, Piyus Kedia, and Rahul Purandare. 2022. The road not taken: exploring alias analysis based optimizations missed by the compiler. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 153 (Oct. 2022), 25 pages. doi:10.1145/3563316
 - [16] Jong-Deok Choi, Michael Burke, and Paul Carini. 1993. Efficient Flow-Sensitive Interprocedural Computation of Pointer-Induced Aliases and Side Effects. In *Proceedings of the 20th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Charleston, South Carolina, USA) (POPL '93). Association for Computing Machinery, New York, NY, USA, 232–245. doi:10.1145/158511.158639
 - [17] R. Chowdhury, Peter Djeu, B. Cahoon, J. Burrill, and K. McKinley. 2004. The Limits of Alias Analysis for Scalar Optimizations. In *CC*.
 - [18] K. D. Cooper and K. Kennedy. 1989. Fast Interprocedural Alias Analysis. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Austin, Texas, USA) (POPL '89). Association for Computing Machinery, New York, NY, USA, 49–59. doi:10.1145/75277.75282
 - [19] Jeff Da Silva and J. Gregory Steffan. 2006. A Probabilistic Pointer Analysis for Speculative Optimizations. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems* (San Jose, California, USA) (ASPLOS XII). Association for Computing Machinery, New York, NY, USA, 416–425. doi:10.1145/1168857.1168908
 - [20] Manuvir Das. 2000. Unification-based pointer analysis with directional assignments. In *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation* (Vancouver, British Columbia, Canada) (PLDI '00). Association for Computing Machinery, New York, NY, USA, 35–46. doi:10.1145/349299.349309
 - [21] Amer Diwan, Kathryn S. McKinley, and J. Eliot B. Moss. 1998. Type-Based Alias Analysis. In *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation* (Montreal, Quebec, Canada) (PLDI '98). Association for Computing Machinery, New York, NY, USA, 106–117. doi:10.1145/277650.277670
 - [22] Maryam Emami, Rakesh Ghiya, and Laurie J. Hendren. 1994. Context-Sensitive Interprocedural Points-to Analysis in the Presence of Function Pointers. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation* (Orlando, Florida, USA) (PLDI '94). Association for Computing Machinery, New York, NY, USA, 242–256. doi:10.1145/178243.178264
 - [23] Manel Fernández and Roger Espasa. 2002. Speculative Alias Analysis for Executable Code. In *Proceedings of the 2002 International Conference on Parallel Architectures and Compilation Techniques (PACT '02)*. IEEE Computer Society, USA, 222–231. doi:10.1109/PACT.2002.1106020
 - [24] Susan L. Graham, Peter B. Kessler, and Marshall K. McKusick. 1982. Gprof: A Call Graph Execution Profiler. In *Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction* (Boston, Massachusetts, USA) (SIGPLAN '82). Association for Computing Machinery, New York, NY, USA, 120–126. doi:10.1145/800230.806987
 - [25] Ben Hardekopf and Calvin Lin. 2007. The Ant and the Grasshopper: Fast and Accurate Pointer Analysis for Millions of Lines of Code. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Diego, California, USA) (PLDI '07). Association for Computing Machinery, New York, NY, USA, 290–299. doi:10.1145/1250734.1250767
 - [26] Ben Hardekopf and Calvin Lin. 2007. Exploiting pointer and location equivalence to optimize pointer analysis. In *Proceedings of the 14th International Conference on Static Analysis* (Kongens Lyngby, Denmark) (SAS'07). Springer-Verlag, Berlin, Heidelberg, 265–280.
 - [27] Ben Hardekopf and Calvin Lin. 2009. Semi-Sparse Flow-Sensitive Pointer Analysis. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Savannah, GA, USA) (POPL '09). Association for Computing Machinery, New York, NY, USA, 226–238. doi:10.1145/1480881.1480911
 - [28] Ben Hardekopf and Calvin Lin. 2011. Flow-Sensitive Pointer Analysis for Millions of Lines of Code. In *Proceedings of the 9th Annual IEEE/ACM International Symposium on Code Generation and Optimization* (CGO '11). IEEE Computer Society, USA, 289–298. doi:10.1109/CGO.2011.5764696
 - [29] Nevin Heintze and Olivier Tardieu. 2001. Ultra-fast aliasing analysis using CLA: a million lines of C code in a second. In *Proceedings of the ACM SIGPLAN 2001 Conference on Programming Language Design and Implementation* (Snowbird, Utah, USA) (PLDI '01). Association for Computing Machinery, New York, NY, USA, 254–263. doi:10.1145/378795.378855
 - [30] Michael Hind. 2001. Pointer Analysis: Haven't We Solved This Problem Yet?. In *Proceedings of the 2001 ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering* (Snowbird, Utah, USA) (PASTE '01). Association for Computing Machinery, New York, NY, USA, 54–61. doi:10.1145/379605.379665
 - [31] Michael Hind, Michael Burke, Paul Carini, and Jong-Deok Choi. 1999. Interprocedural Pointer Alias Analysis. *ACM Trans. Program. Lang. Syst.* 21, 4 (July 1999), 848–894. doi:10.1145/325478.325519
 - [32] A. S. Huang, G. Slavenburg, and J. P. Shen. 1994. Speculative Disambiguation: A Compilation Technique for Dynamic Memory Disambiguation. In *Proceedings of the 21st Annual International Symposium on Computer Architecture* (Chicago, Illinois, USA) (ISCA '94). IEEE Computer Society Press, Washington, DC, USA, 200–210. doi:10.1145/

- 192007.192012
- [33] Jan Hueckelheim and Johannes Doerfert. 2023. ORAQL — Optimistic Responses to Alias Queries in LLVM. In *Proceedings of the 52nd International Conference on Parallel Processing* (Salt Lake City, UT, USA) (ICPP '23). Association for Computing Machinery, New York, NY, USA, 655–664. doi:10.1145/3605573.3605644
 - [34] S. Jaiswal, Uday P. Khedker, and Supratik Chakraborty. 2018. Demand-driven Alias Analysis : Formalizing Bidirectional Analyses for Soundness and Precision. *ArXiv abs/1802.00932* (2018). <https://arxiv.org/abs/1802.00932>
 - [35] Rick Kufrin. 2005. Perfsuite: An Accessible, Open Source Performance Analysis Environment for Linux. In *In Proc. of the Linux Cluster Conference, Chapel*.
 - [36] William Landi and Barbara G. Ryder. 1992. A Safe Approximate Algorithm for Interprocedural Aliasing. In *Proceedings of the ACM SIGPLAN 1992 Conference on Programming Language Design and Implementation* (San Francisco, California, USA) (PLDI '92). Association for Computing Machinery, New York, NY, USA, 235–248. doi:10.1145/143095.143137
 - [37] Chris Lattner, Andrew Lenharth, and Vikram Adve. 2007. Making Context-Sensitive Points-to Analysis with Heap Cloning Practical for the Real World. In *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation* (San Diego, California, USA) (PLDI '07). Association for Computing Machinery, New York, NY, USA, 278–289. doi:10.1145/1250734.1250766
 - [38] Jin Lin, Tong Chen, Wei-Chung Hsu, Pen-Chung Yew, Roy Dz-Ching Ju, Tin-Fook Ngai, and Sun Chan. 2003. A Compiler Framework for Speculative Analysis and Optimizations. In *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation* (San Diego, California, USA) (PLDI '03). Association for Computing Machinery, New York, NY, USA, 289–299. doi:10.1145/781131.781164
 - [39] John Lu and Keith D. Cooper. 1997. Register Promotion in C Programs. In *Proceedings of the ACM SIGPLAN 1997 Conference on Programming Language Design and Implementation* (Las Vegas, Nevada, USA) (PLDI '97). Association for Computing Machinery, New York, NY, USA, 308–319. doi:10.1145/258915.258943
 - [40] Markus Mock, Manuvir Das, Craig Chambers, and Susan J. Eggers. 2001. Dynamic Points-to Sets: A Comparison with Static Analyses and Potential Applications in Program Understanding and Optimization. In *Proceedings of the 2001 ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering* (Snowbird, Utah, USA) (PASTE '01). Association for Computing Machinery, New York, NY, USA, 66–72. doi:10.1145/379605.379671
 - [41] Dorit Naishlos. 2004. Autovectorization in GCC. In *Proceedings of the 2004 GCC Developers Summit*. 105–118.
 - [42] Rupesh Nasre, Kaushik Rajan, R. Govindarajan, and Uday Khedker. 2009. Scalable Context-Sensitive Points-to Analysis Using Multi-dimensional Bloom Filters. 47–62. doi:10.1007/978-3-642-10672-9_6
 - [43] Erik Nystrom, Hong-Seok Kim, and Wen-mei Hwu. 2004. Bottom-Up and Top-Down Context-Sensitive Summary-Based Pointer Analysis. 165–180. doi:10.1007/978-3-540-27864-1_14
 - [44] ERIK MATTHEW NYSTROM. 2012. FULCRA Pointer Analysis Framework. University of Illinois. http://impact.crhc.illinois.edu/paper_details.aspx?paper_id=34
 - [45] Tapti Palit and Pedro Fonseca. 2024. Kaleidoscope: Precise Invariant-Guided Pointer Analysis. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 561–576. doi:10.1145/3620666.3651340
 - [46] David J. Pearce, Paul H.J. Kelly, and Chris Hankin. 2007. Efficient Field-Sensitive Pointer Analysis of C. *ACM Trans. Program. Lang. Syst.* 30, 1 (Nov. 2007), 4–es. doi:10.1145/1290520.1290524
 - [47] David J. Pearce, Paul H. J. Kelly, and Chris Hankin. 2004. Online Cycle Detection and Difference Propagation: Applications to Pointer Analysis. *Software Quality Journal* 12, 4 (Dec. 2004), 311–337. doi:10.1023/B:SQJO.0000039791.93071.a2
 - [48] Fernando Magno Quintao Pereira and Daniel Berlin. 2009. Wave Propagation and Deep Propagation for Pointer Analysis. In *Proceedings of the 7th Annual IEEE/ACM International Symposium on Code Generation and Optimization (CGO '09)*. IEEE Computer Society, USA, 126–135. doi:10.1109/CGO.2009.9
 - [49] Ankush Phulia, Vaibhav Bhagee, and Sorav Bansal. 2020. OOEla: Order-of-Evaluation Based Alias Analysis for Compiler Optimization. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (PLDI 2020). Association for Computing Machinery, New York, NY, USA, 839–853. doi:10.1145/3385412.3385962
 - [50] B. K. Rosen, M. N. Wegman, and F. K. Zadeck. 1988. Global Value Numbers and Redundant Computations. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '88). Association for Computing Machinery, New York, NY, USA, 12–27. doi:10.1145/73560.73562
 - [51] Marc Shapiro and Susan Horwitz. 1997. Fast and accurate flow-insensitive points-to analysis. In *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Paris, France) (POPL '97). Association for Computing Machinery, New York, NY, USA, 1–14. doi:10.1145/263699.263703
 - [52] Victor Hugo Sperle Campos, Péricles Rafael Alves, Henrique Nazaré Santos, and Fernando Magno Quintão Pereira. 2016. Restrictification of Function Arguments. In *Proceedings of the 25th International Conference on Compiler Construction* (Barcelona, Spain) (CC 2016). Association for Computing Machinery, New York, NY, USA, 163–173. doi:10.1145/2892208.2892225
 - [53] Bjarne Steensgaard. 1996. Points-to Analysis in Almost Linear Time. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (St. Petersburg Beach, Florida, USA) (POPL '96). Association for Computing Machinery, New York, NY, USA, 32–41. doi:10.1145/237721.237727
 - [54] Yulei Sui, Peng Di, and Jingling Xue. 2016. Sparse flow-sensitive pointer analysis for multithreaded programs. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization* (Barcelona, Spain) (CGO '16). Association for Computing Machinery, New York, NY, USA, 160–170. doi:10.1145/2854038.2854043
 - [55] Yulei Sui and Jingling Xue. 2016. SVF: Interprocedural Static Value-Flow Analysis in LLVM. In *Proceedings of the 25th International Conference on Compiler Construction* (Barcelona, Spain) (CC 2016). Association for Computing Machinery, New York, NY, USA, 265–266. doi:10.1145/2892208.2892235
 - [56] Yulei Sui, Sen Ye, Jingling Xue, and Jie Zhang. 2014. Making context-sensitive inclusion-based pointer analysis practical for compilers using parameterised summarisation. *Softw. Pract. Exper.* 44, 12 (Dec. 2014), 1485–1510. doi:10.1002/spe.2214
 - [57] Teck Bok Tok, Samuel Z. Guyer, and Calvin Lin. 2006. Efficient flow-sensitive interprocedural data-flow analysis in the presence of pointers. In *Proceedings of the 15th International Conference on Compiler Construction* (Vienna, Austria) (CC'06). Springer-Verlag, Berlin, Heidelberg, 17–31. doi:10.1007/11688839_3
 - [58] Jiayi Wang, Yu Wang, Ke Wang, and Linzhang Wang. 2025. SILVA: A Scalable Incremental Layered Sparse Value-Flow Analysis. *ACM Trans. Softw. Eng. Methodol.* (March 2025). doi:10.1145/3725214 Just Accepted.
 - [59] Michel Weber, Theodoros Theodoridis, and Zhendong Su. 2025. Relaxing Alias Analysis: Exploring the Unexplored Space. *Proc. ACM Program. Lang.* 9, PLDI, Article 155 (June 2025), 24 pages. doi:10.1145/3729254
 - [60] John Whaley and Monica S. Lam. 2004. Cloning-Based Context-Sensitive Pointer Alias Analysis Using Binary Decision Diagrams. In *Proceedings of the ACM SIGPLAN 2004 Conference on Programming*

- Language Design and Implementation* (Washington DC, USA) (*PLDI '04*). Association for Computing Machinery, New York, NY, USA, 131–144. doi:10.1145/996841.996859
- [61] Robert P. Wilson and Monica S. Lam. 1995. Efficient Context-Sensitive Pointer Analysis for C Programs. In *Proceedings of the ACM SIGPLAN 1995 Conference on Programming Language Design and Implementation* (La Jolla, California, USA) (*PLDI '95*). Association for Computing Machinery, New York, NY, USA, 1–12. doi:10.1145/207110.207111
- [62] Steven Cameron Woo, Moriyoshi Ohara, Evan Torrie, Jaswinder Pal Singh, and Anoop Gupta. 1995. The SPLASH-2 Programs: Characterization and Methodological Considerations. In *Proceedings of the 22nd Annual International Symposium on Computer Architecture* (S. Margherita Ligure, Italy) (*ISCA '95*). Association for Computing Machinery, New York, NY, USA, 24–36. doi:10.1145/223982.223990
- [63] Jingyue Wu, Gang Hu, Yang Tang, and Junfeng Yang. 2013. Effective Dynamic Detection of Alias Analysis Errors. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering* (Saint Petersburg, Russia) (*ESEC/FSE 2013*). Association for Computing Machinery, New York, NY, USA, 279–289. doi:10.1145/2491411.2491439
- [64] Xin Zheng and Radu Rugina. 2008. Demand-Driven Alias Analysis for C. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (*POPL '08*). Association for Computing Machinery, New York, NY, USA, 197–208. doi:10.1145/1328438.1328464
- [65] Jianwen Zhu. 2005. Towards scalable flow and context sensitive pointer analysis. In *Proceedings. 42nd Design Automation Conference, 2005*. IEEE, 831–836. https://doi.org/10.1145/1065579.1065798

Received 2025-11-08; accepted 2025-12-10